

ECE 1387

lecture 10

Nov
21, 2025

-
- partitioning
 - technology mapping

Course paper

25 marks

5 presentation

10 survey

10 critique / critical analysis

Discuss/email topic by
Dec. 1

Refinement

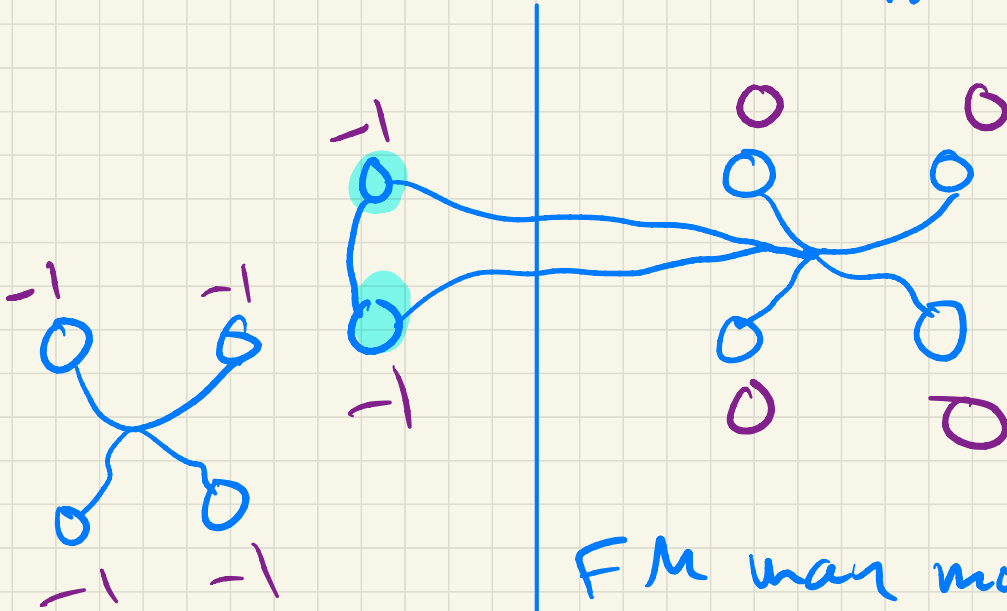
1) FM algorithm

- Early exit FM - Halt a pass if $X\%$ of swaps did not improve crossing count.
Say $X = 1\%$

2) Limit FM to 2 passes.

Limitation of FM

Say balance constraint
is 60/40 split



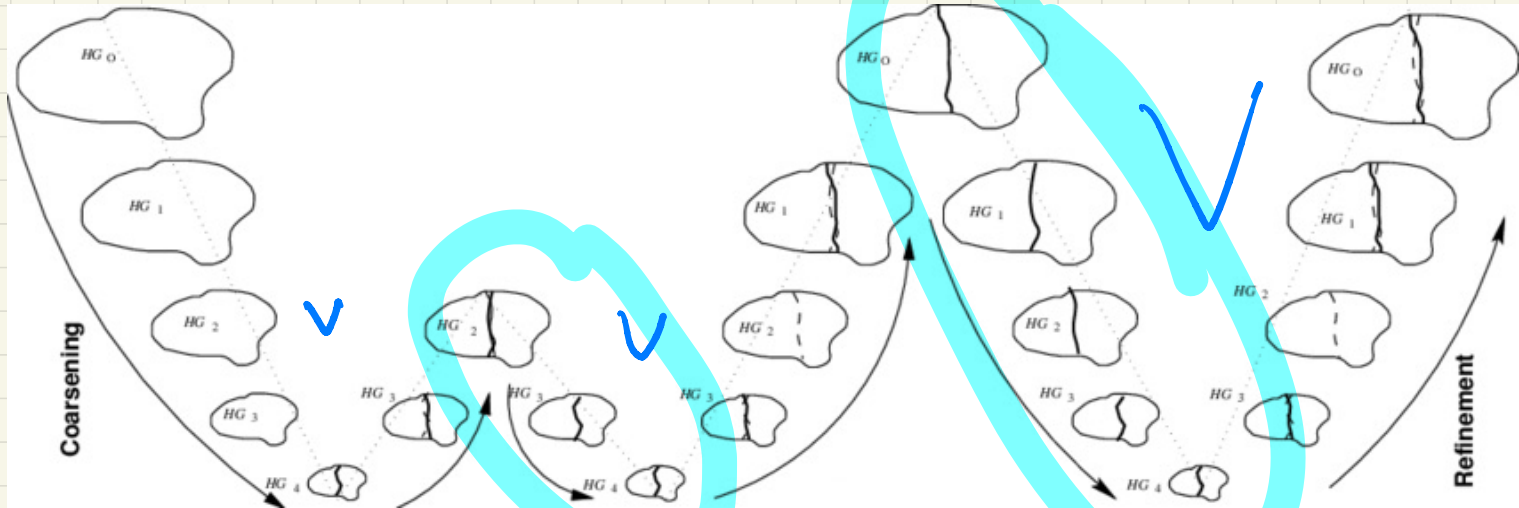
FM can move any
of verts with -1
gain

But what we really want is
to move two  vertices to
other side

Limitation of FM leads to
"hyperedge refinement"

- Randomly visit hyperedges that are cut
- Determine if can move subset of vertices on hyperedge so no larger cut.
- A greedy approach
- This is followed by FM-based approaches

Multi-phase Refinement



V Cycles
V Cycles

restricted coarsening
↳ only vertices in
same partition
can collapse

Technology Mapping

(part of logic synthesis)

Logic synthesis has 2 steps

- 1) tech-independent optimization
 - like k-maps (but in algorithms)
 - FSM optimizations
 - retiming?
 - removing 'dead circuitry'
- 2) tech-mapping
 - optimized netlist is turned into a network of gates from target library

→ don't care optimization

TM problem defⁿ

logic functions

Given: 1) Boolean network $G(V, E)$

2) Library of available gates (standard cells)

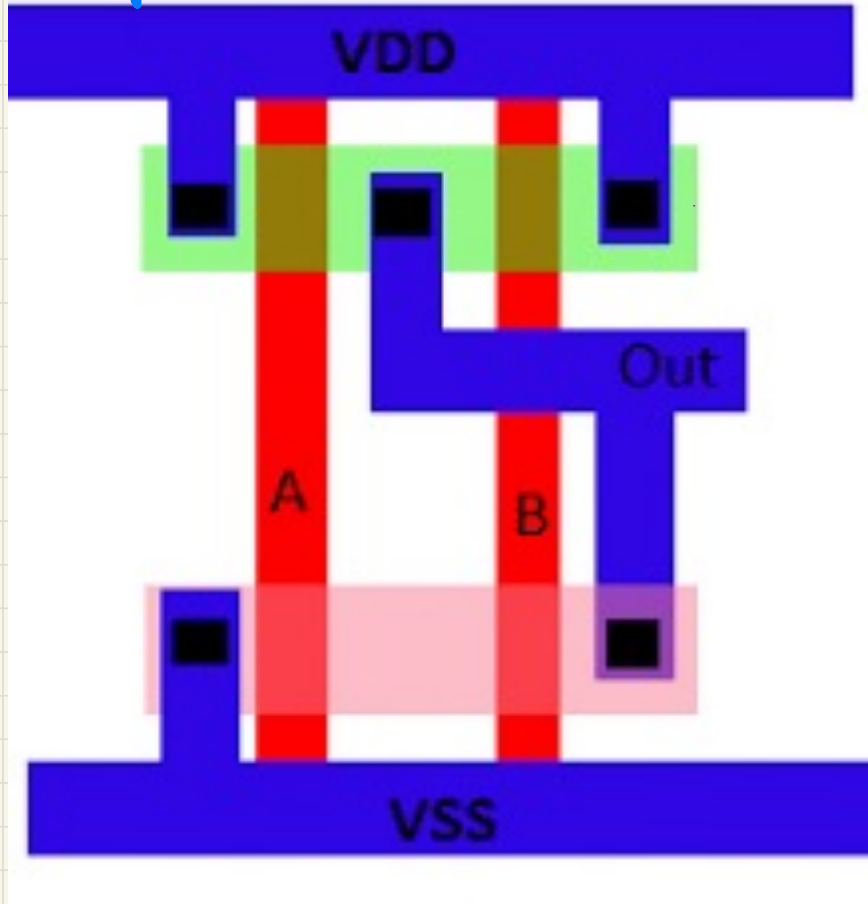
↑ dependencies

Find: A netlist of available gates that implements functionality of G

So as to minimize: area, power, delay, ...

We will consider TM to
ASIC standard cells ...

Example standard cell



<https://teamvlsi.com/2020/05/standard-cells-in-asic-design-standard-cells-in-vlsi.html?amp=1>

NAND-2
standard cell.

All cells have
Same height.

Example Boolean network $y = t(a+g)$

library elem area delay



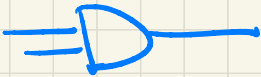
3

1



6

2



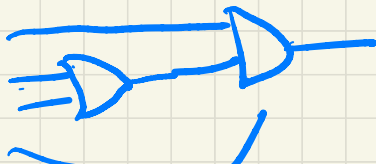
8

2.5



8

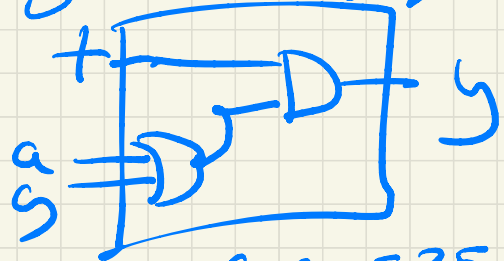
2.5



25

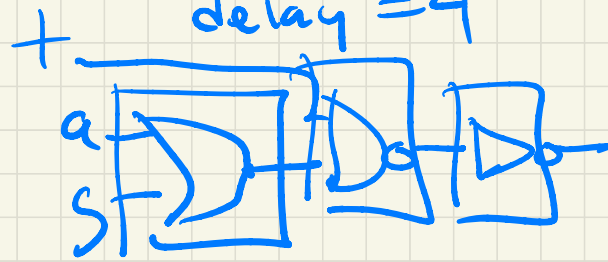
4

OR-AND cell



area = 25

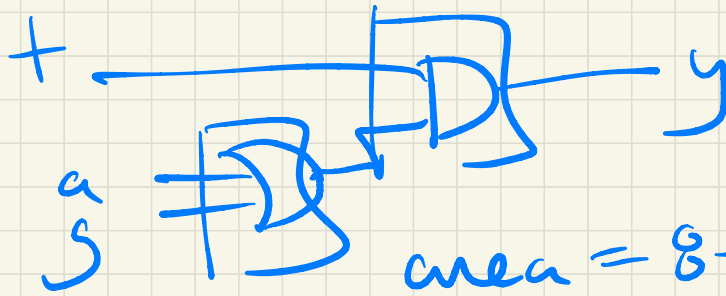
delay = 4



area = 8 + 6 + 3

= 17

delay = 5.5



$$\text{area} = 8 + 8 = 16$$

$$\text{delay} = 2.5 + 2.5 = 5$$

This is a small library $\frac{3}{1}$ BN
Real situation:

- library with dozens or 100+ cells
- user constraints (area, delay)
- larger circuit

DAGON: Technology Binding and Local Optimization by DAG Matching

Kurt Keutzer
AT&T Bell Laboratories
Murray Hill, New Jersey 07974

— UC Berkeley
— CTO Synopsis

ABSTRACT

Technology binding is the process of mapping a technology independent description of a circuit into a particular technology. This paper outlines a formalism of this problem and offers a solution to the problem in terms of matching patterns, describing technology specific cells and optimizations, against a technology independent circuit represented as a directed acyclic graph. This solution is implemented in *DAGON*. *DAGON* rests on a firm algorithmic foundation, and is able to guarantee locally optimal matches against a set of over three thousand patterns. *DAGON* is an integral part of a synthesis system that has been found to provide industrial quality solutions to real circuit design problems.

1. Introduction

technologies. *DAGON* rests on a firm algorithmic foundation, and is able to guarantee locally optimal matches against a set of over three thousand patterns.

2. The Formalism

Before going into how compiler techniques are used in *DAGON* we will first take a step back to view the problem of technology binding and local optimizations under a broad formalism. In this paper we assume that the reader is familiar with basic tree and graph terminology, as described, for instance, in [AHU76]. Any set of boolean functions with common sub-expressions can be viewed as a directed acyclic graph (DAG) in which the labels of the vertices in the graph are the boolean operators AND and OR and edges are labeled either 1, for the true value of the signal associated with the vertex, or 0, for the inverted value. We may think of this DAG as a circuit with edges directed from outputs to inputs. We present here the different stages of the technology binding process.

1. A technology independent graph $G_i = (V, E)$ such that each v_i in V , $\text{label}(v_i) \in \{\text{AND}, \text{OR}\}$ and each e_i in $E \in$

Step 1: Take Boolean network
and rewrite using a set of
"base functions"

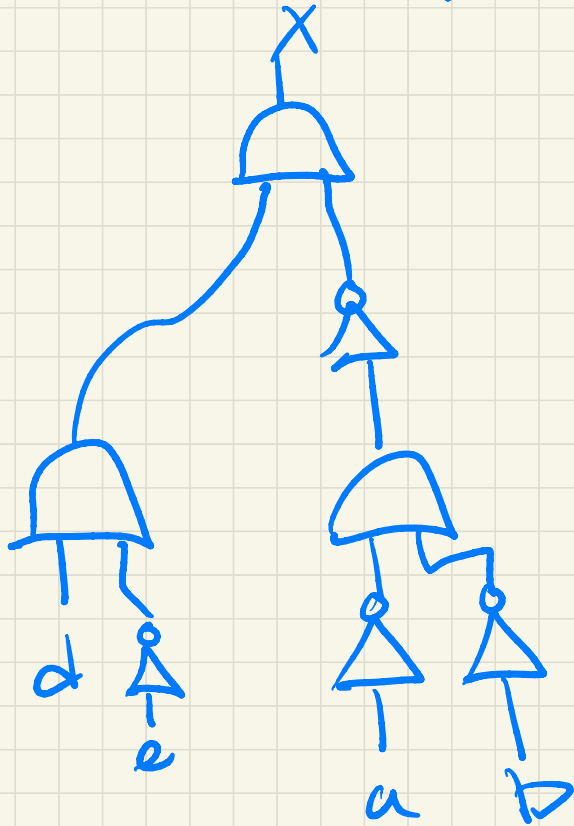
Popular choice for base functions

2-input AND + inverters

AIG: AND-inverter graph

↳ Used in ABC logic synthesis
tool (UC Berkeley)

$x = d\bar{e}(a+b)$ express as AIG



$a+b =$
 $\overline{\overline{a}\overline{b}}$ de Morgans

called "subject
graph"

Subject of our
optimizations

EPFL logic synthesis tool

MockTurtle

— base functions are Majority + inv

MIB : majority inverter
graph

Majority function

$$f = ab + bc + ac \quad \left. \vphantom{f = ab + bc + ac} \right\} \begin{array}{l} \text{Carry function} \\ \text{in Full adder} \end{array}$$

$$\text{if } c = 0, f = ab \equiv \text{AND}$$

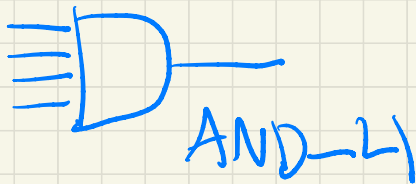
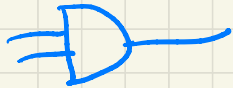
$$c = 1, f = a + b \equiv \underline{\text{OR}}$$

In the work: use MIG to map
regular logic onto ~~FPGA~~
Carry chain.

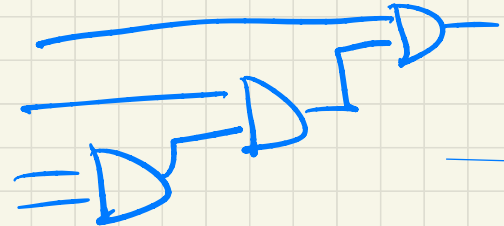
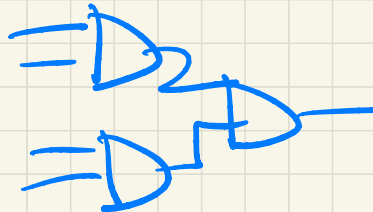
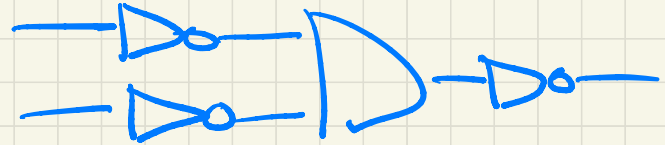
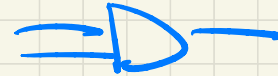
Step 2: Express all gates in
library in same base functions

Examples

gate



Pattern graph



May be multiple pattern graphs
for given gate (like AND-4)

Find all such pattern graphs

Now have subject graph and many pattern graphs.

Because the gates are standard cells,

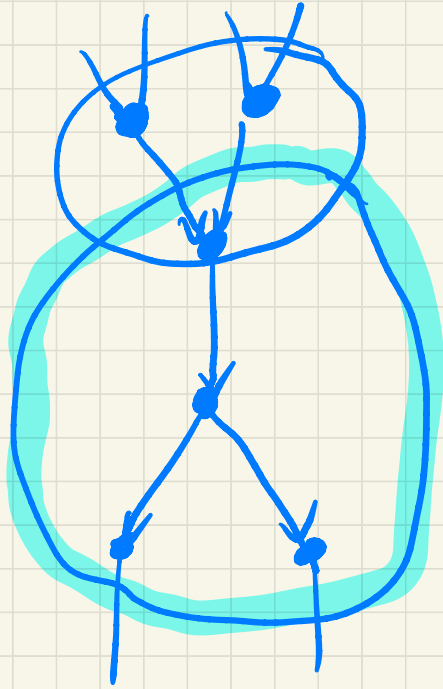
Know: — area of each
— (something about) delay

Pose TM problem as a "covering" problem. Well known problem in graph theory

Defⁿ: A cover of subj. graph by other pattern graphs is a network of pattern graphs such that:

i) Every node of subject graph must be covered by a node of a pattern graph & matches its type.

ii) legal connectivity between pattern graphs : each input required by a pattern graph is produced by an output of another pattern graph in network.



No legal

Since ● needs
inputs that
are not produced
by other pattern
graph outputs

TM is to find a minimum cost cover. Cost: $\text{Area} = \sum \text{Area of gates}$

Delay $\equiv$ max delay along critical path of mapped circuit.

Exact covering problem is very hard NP-hard.

To simplify, DAGON breaks

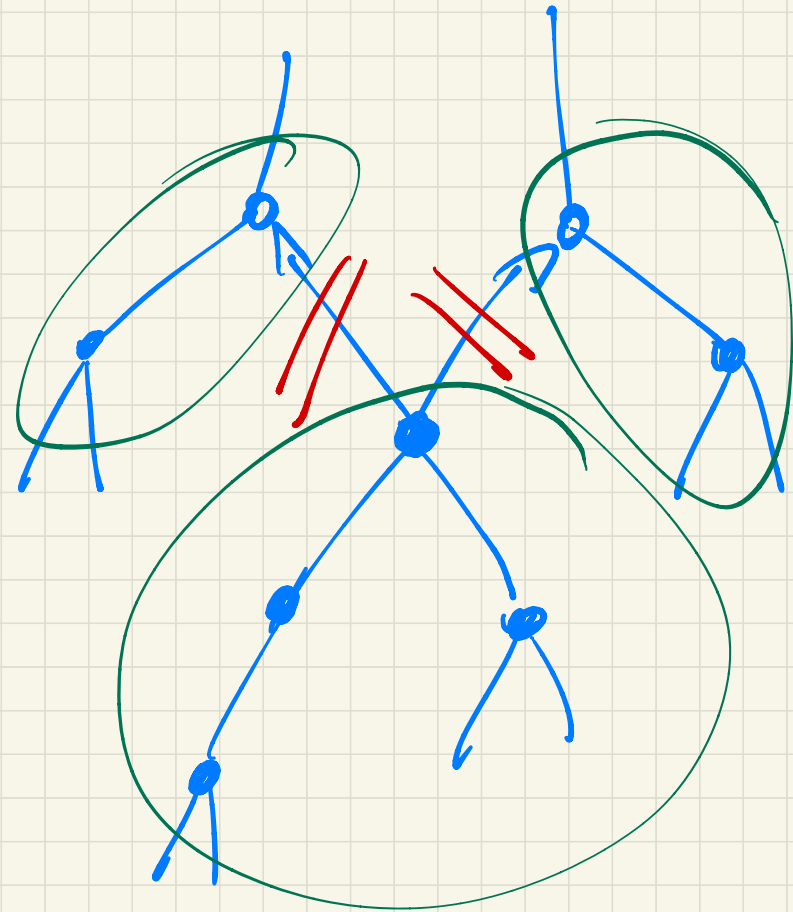
① Subject graph into trees
(all fanout $\equiv 1$)

} not
done
today

② restricts gates to be trees
too

Covers trees with trees, and
glues back together

Network K broke
into 3 trees



How to cover a tree with trees? ^{optimization approach}

Dynamic programming

Applies to problems with two
key ingredients

1) "optimal substructure": overall

solⁿ can be broken into subprobs.

Solve subprobs optimally. Stitch
together to form optimal solⁿ of
overall problem

2) Overlapping sub probs

- In solving overall prob, we "visit" (need to solve) same sub probs many times (can store solⁿ in a table)

Basic idea

1. Break overall prob into subprobs
2. Solve subprobs optimally
3. Use solⁿs to subprobs to compose/construct solⁿ to overall prob.